

مجموعه مسائل پیشرفته: بهینه‌سازی کد با ویژگی‌های NumPy (۲۰ مسئله)

هدف: تمرین مهارت‌های بهینه‌سازی و افزایش کارایی کد پایتون با تکیه بر الگوهای بردارسازی (Vectorization)، پخش‌پذیری (Broadcasting)، نمایه‌گذاری پیشرفته (Advanced Indexing)، عملیات تجمعی (Reductions)، حافظه و چیدمان (Memory/Layout)، و ابزارهای محاسبات عددی NumPy.

پیش‌نیاز: آشنایی خوب با NumPy، محاسبات آرایه‌ای، و مفاهیم پایه پیچیدگی زمانی/حافظه.

قواعد کلی: - در هر مسئله، هدف بازنویسی نسخه کند به نسخه سریع‌تر با استفاده از NumPy است. - تا جای ممکن از حلقه‌های پایتون (for/while) دوری کنید. - به تخصیص حافظه (allocation) و کپی ناخواسته (copy) حساس باشید. - اگر از ترفندهای ظریف استفاده می‌کنید (مثل `as_strided`)، ریسک‌ها را توضیح دهید.

۱) فاصله جفتی (Pairwise Distances) با بردارسازی

صورت مسئله

دو ماتریس X و Y به شکل (n, d) و (m, d) دارید. می‌خواهید ماتریس فواصل اقلیدسی مربعی بین همه جفت‌ها را بسازید:

$$D_{ij} = \|X_i - Y_j\|^2$$

کد کند

```
import numpy as np

def pairwise_sqdist_slow(X, Y):
    n, d = X.shape
    m, _ = Y.shape
    D = np.empty((n, m), dtype=float)
    for i in range(n):
```

```

        :for j in range(m)
            s = 0.0
            :for k in range(d)
diff = X[i, k] - Y[j, k]
            s += diff * diff
            D[i, j] = s
        return D

```

خواسته‌ها

- نسخهٔ سریع با استفاده از broadcasting یا هویت جبری
- $|x-y|^2 = |x|^2 + |y|^2 - 2x \cdot y$ بنویسید.
- از ایجاد آرایه‌های میانی بسیار بزرگ (مثل (n, m, d)) تا حد امکان اجتناب کنید.
- پیچیدگی حافظهٔ راه‌حل را تحلیل کنید.

۲) محاسبهٔ میانگین متحرک (Moving Average) بدون حلقه

صورت مسئله

آرایهٔ یک‌بعدی a و پنجرهٔ w دارید. میانگین متحرک با پنجرهٔ ثابت را محاسبه کنید.

کد کند

```

import numpy as np

def moving_avg_slow(a, w):
    out = np.empty(len(a) - w + 1, dtype=float)
    :for i in range(len(out))
        out[i] = a[i:i+w].mean
    return out

```

خواسته‌ها

- نسخهٔ سریع با `cumsum` یا `stride tricks` بنویسید.
- تفاوت `np.lib.stride_tricks.sliding_window_view` با `as_strided` را توضیح دهید.

۳) نرمال‌سازی سطرها با جلوگیری از تقسیم بر صفر

صورت مسئله

ماتریس X با شکل (n, d) دارید. می‌خواهید هر سطر را بر نرم $L2$ خودش تقسیم کنید. اگر نرم صفر بود، سطر را صفر نگه دارید.

کد کند

```
import numpy as np

def row_normalize_slow(X):
    Xn = X.copy().astype(float)
    for i in range(Xn.shape[0]):
        norm = np.sqrt((Xn[i] ** 2).sum())
        if norm != 0:
            Xn[i] /= norm
        else:
            Xn[i] = 0
    return Xn
```

خواسته‌ها

- نسخهٔ سریع با `np.linalg.norm(..., axis=1, keepdims=True)` و `masking` بنویسید.
- از `in-place` بودن و اثرش بر حافظه صحبت کنید.

۴) گروه‌بندی و جمع‌زدن (GroupBy Sum) با

`np.add.at`

صورت مسئله

آرایهٔ `values` و برچسب‌های صحیح `labels` به طول `N` دارید، و می‌خواهید مجموع هر گروه را بسازید.

کد کند

```
import numpy as np

def group_sum_slow(values, labels, n_groups):
    out = np.zeros(n_groups, dtype=values.dtype)
    for v, g in zip(values, labels):
        out[g] += v
    return out
```

خواسته‌ها

- نسخهٔ سریع با `np.bincount` یا `np.add.at` بنویسید.
- دربارهٔ تفاوت‌های کارایی و کاربردهای هرکدام بحث کنید.

۵) هیستوگرام دوبعدی بدون حلقه

صورت مسئله

بردارهای `x` و `y` و تعداد سطل‌ها `bins` دارید. می‌خواهید هیستوگرام ۲بعدی بسازید.

کد کند

```
import numpy as np

def hist2d_slow(x, y, bins, xmin, xmax, ymin, ymax):
    H = np.zeros((bins, bins), dtype=int)
    for xi, yi in zip(x, y):
        bx = int((xi - xmin) / (xmax - xmin) * bins)
        by = int((yi - ymin) / (ymax - ymin) * bins)
        if 0 <= bx < bins and 0 <= by < bins:
            H[by, bx] += 1
    return H
```

خواسته‌ها

- نسخهٔ سریع با `np.histogram2d` یا تبدیل شاخص‌ها و `np.add.at` ارائه دهید.
- به برخورد با نقاط مرزی و clipping توجه کنید.

۶) فیلتر شرطی و جایگزینی (Where vs Mask)

صورت مسئله

آرایهٔ `a` دارید. می‌خواهید مقادیر خارج از بازه `[lo, hi]` را به نزدیک‌ترین کران `clamp` کنید.

کد کند

```
import numpy as np

def clamp_slow(a, lo, hi):
    out = a.copy()
    for i in range(len(out)):
        if out[i] < lo:
```

```
out[i] = lo
:elif out[i] > hi
out[i] = hi
return out
```

خواسته‌ها

- نسخهٔ سریع با `np.clip` و نسخهٔ دیگر با `np.where` بنویسید.
- تفاوت ایجاد کپی و in-place را بررسی کنید.

۷) محاسبهٔ همبستگی ستون‌ها با حذف میانگین

صورت مسئله

ماتریس `X` با شکل `(n, d)` دارید. می‌خواهید ماتریس همبستگی ستون‌ها را محاسبه کنید.

کد کند

```
import numpy as np

def corr_slow(X):
    n, d = X.shape
    C = np.empty((d, d), float)
    for i in range(d):
        xi = X[:, i] - X[:, i].mean()
        for j in range(d):
            xj = X[:, j] - X[:, j].mean()
            C[i, j] = (xi * xj).sum() / (n - 1)
    return C
```

خواسته‌ها

- نسخهٔ سریع با مرکزسازی یکباره و ضرب ماتریسی (مثلاً `Xc.T @ Xc`) بنویسید.

• در صورت نیاز، نرمال‌سازی برای همبستگی (نه کوواریانس) را اضافه کنید.

۸) انتخاب Top-k در هر سطر بدون مرتب‌سازی کامل

صورت مسئله

ماتریس `A` با شکل `(n, m)` دارید. برای هر سطر، اندیس `k` مقدار بزرگتر را بیابید.

کد کند

```
import numpy as np

def topk_idx_slow(A, k):
    idx = np.empty((A.shape[0], k), dtype=int)
    for i in range(A.shape[0]):
        idx[i] = np.argsort(A[i])[::-1][:k]
    return idx
```

خواسته‌ها

- نسخهٔ سریع با `np.argpartition` بنویسید.
- تفاوت `argpartition` و `argsort` را از نظر پیچیدگی توضیح دهید.

۹) تجمیع پنجره‌ای دوبعدی با `sliding_window_view`

صورت مسئله

تصویر خاکستری `img` با شکل `(H, W)` دارید. می‌خواهید میانگین هر پنجرهٔ `k×k` را بسازید (`valid`).

کد کند

```
import numpy as np

def mean_filter_slow(img, k)
    H, W = img.shape
    out = np.empty((H-k+1, W-k+1), float)
    for i in range(H-k+1)
    for j in range(W-k+1)
        out[i, j] = img[i:i+k, j:j+k].mean
    return out
```

خواسته‌ها

- نسخهٔ سریع با `sliding_window_view` و `reduction` روی محورهای مناسب بنویسید.
- هزینهٔ حافظهٔ `view` را نسبت به کپی توضیح دهید.

۱۰) تبدیل فهرست اندیس‌ها به ماتریس پراکندهٔ شمارشی

صورت مسئله

دو آرایهٔ `rows` و `cols` به طول `N` دارید که مختصات سلول‌های غیرصفر در یک ماتریس پراکنده را مشخص می‌کنند. می‌خواهید یک ماتریس شمارشی (`count`) ایجاد کنید که تعداد دفعات تکرار هر جفت `(row, col)` را در ماتریس اصلی نشان دهد.

کد کند

```
import numpy as np

def count_sparse_slow(rows, cols, shape)
    {} = counts
    for r, c in zip(rows, cols)
```



```

        key = (r, c)
        counts[key] = counts.get(key, 0) + 1

        mat = np.zeros(shape, dtype=int)
        for (r, c), count in counts.items():
            if r < shape[0] and c < shape[1]:
                mat[r, c] = count
        return mat

```

خواسته‌ها

- نسخهٔ سریع با استفاده از `np.add.at` یا `np.bincount` (با تبدیل مختصات به اندیس یک‌بعدی) بنویسید.
- محدودیت‌های `np.bincount` در مورد مقادیر منفی یا غیر صحیح برچسب‌ها را ذکر کنید.

۱۱) محاسبهٔ فاصلهٔ منهتن (Manhattan Distance) بین نقاط

صورت مسئله

دو آرایهٔ `A` و `B` به شکل `(n, d)` دارید که هر سطر نشان‌دهندهٔ یک نقطه در فضای `d` بعدی است. می‌خواهید ماتریس فواصل منهتن بین تمام جفت نقاط را محاسبه کنید:

$$D_{ij} = \sum_{k=1}^d |A_{ik} - B_{jk}|$$

کد کند

```

import numpy as np

def manhattan_dist_slow(A, B):
    n = A.shape[0]

```

```

        m = B.shape[0]
        d = A.shape[1]
    D = np.empty((n, m), dtype=float)
        :for i in range(n)
            :for j in range(m)
                dist = 0.0
                :for k in range(d)
                    dist += abs(A[i, k] - B[j, k])
                    D[i, j] = dist
    return D

```

خواسته‌ها

- نسخهٔ سریع با استفاده از broadcasting و `np.sum` روی محور مناسب بنویسید.
- نشان دهید که چگونه می‌توان این را با `np.abs` و `np.sum(..., axis=...)` انجام داد.

۱۲) یافتن نقاط خارج از یک دایره (Circle Clipping)

صورت مسئله

مجموعه‌ای از نقاط (x, y) در یک آرایه P با شکل $(N, 2)$ دارید. می‌خواهید تمام نقاطی را که در داخل یک دایره با مرکز (cx, cy) و شعاع r قرار دارند، پیدا کنید.

کد کند

```

import numpy as np

def points_in_circle_slow(P, cx, cy, r)
    [] = inside
    :for i in range(P.shape[0])
        x, y = P[i]
        dist_sq = (x - cx)**2 + (y - cy)**2
        :if dist_sq <= r**2

```

```
inside.append(P[i])
return np.array(inside)
```

خواسته‌ها

- نسخهٔ سریع با استفاده از broadcasting و عملیات شرطی بنویسید.
- از ایجاد لیست‌های پایتون تا حد امکان اجتناب کنید.

۱۳) محاسبهٔ کوواریانس سفارشی (Custom Covariance)

صورت مسئله

ماتریس X با شکل (n, d) دارید. می‌خواهید ماتریس کوواریانس ستون‌ها را محاسبه کنید. این بار، فرض کنید میانگین هر ستون از قبل از آن کم شده است.

کد کند

```
import numpy as np

def custom_cov_slow(X_centered):
    n, d = X_centered.shape
    C = np.empty((d, d), float)
    for i in range(d):
        for j in range(d):
            C[j, i] = (X_centered[:, i] * X_centered[:, j]).sum() / (n - 1)
    return C
```

خواسته‌ها

- نسخهٔ سریع با ضرب ماتریسی $X_centered.T @ X_centered$ بنویسید.
- نقش $n-1$ را در محاسبهٔ کوواریانس توضیح دهید.

۱۴) شمارش مقادیر منحصر به فرد در پنجره (Unique Counts in Window)

صورت مسئله

آرایه یک بعدی `a` و اندازه پنجره `w` دارید. برای هر پنجره متحرک، تعداد مقادیر منحصر به فرد را بشمارید.

کد کند

```
import numpy as np

def unique_counts_slow(a, w):
    out = np.empty(len(a) - w + 1, dtype=int)
    for i in range(len(out)):
        window = a[i:i+w]
        out[i] = len(np.unique(window))
    return out
```

خواسته‌ها

- نسخه سریع با استفاده از `np.add.at` یا `np.bincount` و `sliding_window_view` بنویسید.
 - این مسئله نسبت به میانگین متحرک پیچیده‌تر است؛ راهنمایی: استفاده از دیکشنری‌های موقت در هر پنجره یا روش‌های پیشرفته‌تر.
-

۱۵) مساحت سطوح (Surface Area) مثلث‌ها

صورت مسئله

دو آرایه $v1$ و $v2$ به شکل $(n, 3)$ دارید که هر سطر نشان‌دهنده دو راس یک ضلع مثلث است. رأس سوم هر مثلث از مبدأ $(0, 0, 0)$ فرض می‌شود. می‌خواهید مساحت هر مثلث را محاسبه کنید.

کد کند

```
import numpy as np

def triangle_area_slow(v1, v2):
    n = v1.shape[0]
    areas = np.empty(n, dtype=float)
    for i in range(n):
        a = v1[i]
        b = v2[i]
        ||Area = 0.5 * ||a x b #
        cross_prod = np.cross(a, b)
        areas[i] = 0.5 * np.linalg.norm(cross_prod)
    return areas
```

خواسته‌ها

- نسخه سریع با استفاده از `np.cross` و `np.linalg.norm` به صورت برداری بنویسید.
- مطمئن شوید که `np.linalg.norm` روی محور صحیح اعمال می‌شود.

۱۶) تقاطع آرایه‌ها با شرایط پیچیده

صورت مسئله

دو آرایه A و B به شکل (n, d) دارید. می‌خواهید جفت‌های سطری $(A[i], B[j])$ را پیدا کنید که در شرط زیر صدق می‌کنند:

$$\sum_{k=0}^{d-1} (A_{ik} - B_{jk})^2 < \epsilon$$

یعنی نقاطی که به هم نزدیک هستند.

کد کند

```
import numpy as np

def find_close_pairs_slow(A, B, epsilon):
    n = A.shape[0]
    m = B.shape[0]
    close_pairs = []
    for i in range(n):
        for j in range(m):
            if np.sum((A[i] - B[j])**2) < epsilon:
                close_pairs.append((i, j))
    return np.array(close_pairs)
```

خواسته‌ها

- نسخهٔ سریع با استفاده از broadcasting برای محاسبهٔ ماتریس فواصل مربعی و سپس فیلتر کردن آن بنویسید.
- از ایجاد ماتریس کامل (n, m, d) خودداری کنید.

۱۷) شمارش وقوع هر سطر (Row Counts)

صورت مسئله

ماتریس X با شکل (N, d) دارید. می‌خواهید تعداد دفعات تکرار هر سطر منحصر به فرد را در ماتریس بشمارید.

کد کند

```
import numpy as np

def row_counts_slow(X):
    unique_rows, counts = np.unique(X, axis=0, return_counts=True)
    return unique_rows, counts
```

خواسته‌ها

- نسخهٔ سریع‌تر با استفاده از تبدیل سطرهای به اندیس‌های یک‌بعدی (مثلاً با استفاده از `np.lexsort` یا `hashing`) و سپس `np.bincount` بنویسید.
- نکته: `np.unique` برای این منظور نسبتاً بهینه است، اما هدف تمرین روش‌های دیگر است.

۱۸) جایگزینی مقادیر بر اساس موقعیت (Fancy Indexing for Replacement)

صورت مسئله

آرایه A و دو آرایه `row_indices` و `col_indices` به طول k دارید. می‌خواهید مقادیر `A[row_indices[i], col_indices[i]]` را با مقادیر آرایه `new_values` به طول k جایگزین کنید.

کد کند

```
import numpy as np

def replace_fancy_slow(A, row_indices, col_indices, new_values):
    A_copy = A.copy()
    for i in range(len(row_indices)):
        A_copy[row_indices[i], col_indices[i]] = new_values[i]
    return A_copy
```

خواسته‌ها

- نسخهٔ سریع با استفاده از fancy indexing در NumPy بنویسید.
- نحوهٔ کارکرد fancy indexing را با آرایه‌های اندیس توضیح دهید.

۱۹) محاسبهٔ مقادیر در یک شبکهٔ دلخواه (Interpolation on a Grid)

صورت مسئله

تابع $f(x, y)$ را داریم که فقط در نقاط گسسته $(x_{\text{grid}}, y_{\text{grid}})$ با مقادیر z_{grid} محاسبه شده است. می‌خواهید مقدار تابع را در نقاط دلخواه $(x_{\text{points}}, y_{\text{points}})$ درون محدودهٔ z_{grid} با استفاده از درون‌یابی خطی (bilinear interpolation) محاسبه کنید.

کد کند

```
import numpy as np
from scipy.interpolate import griddata

def interpolate_grid_slow(x_grid, y_grid, z_grid, x_points, y_points):
    points = np.vstack((x_grid.ravel(), y_grid.ravel())).T
    values = z_grid.ravel()
```



```
grid_points = np.vstack((x_points, y_points)).T
```

```
# این تابع از scipy استفاده می‌کند و کند است اگر بخواهیم آن را دستی  
# هدف این است که با broadcasting و fancy indexing به نتیجه‌ای مشابه  
# اما پیاده‌سازی دستی bilinear interpolation پیچیده است.  
# این مسئله بیشتر برای تمرین درک نحوهٔ دستیابی به برداری‌سازی است.  
return griddata(points, values, grid_points, method='linear')
```

خواسته‌ها

- نسخهٔ سریع با استفاده از broadcasting و توابع ریاضی NumPy برای پیاده‌سازی تقریبی bilinear interpolation (یا تمرکز بر یافتن نزدیکترین نقاط و وزن‌دهی ساده) بنویسید.
- نکته: پیاده‌سازی کامل bilinear interpolation با NumPy بدون استفاده از کتابخانه‌های تخصصی پیچیده است. هدف، نشان دادن چگونگی استفاده از broadcasting برای محاسبهٔ فواصل و وزن‌ها است.

۲۰ ضرب ماتریس پراکنده در بردار (Sparse Matrix-Vector Multiplication)

صورت مسئله

یک ماتریس پراکنده M (مثلاً به صورت $(row_indices, col_indices, data)$) و یک بردار v دارید. می‌خواهید حاصل ضرب $M @ v$ را محاسبه کنید.

کد کند

```
import numpy as np

def spmv_slow(row_indices, col_indices, data, v, shape):
    n, m = shape
    result = np.zeros(n, dtype=data.dtype)
    for i in range(len(data)):
        row = row_indices[i]
```

```
col = col_indices[i]
value = data[i]
:if row < n and col < m
result[row] += value * v[col]
return result
```

خواسته‌ها

- نسخهٔ سریع با استفاده از `np.add.at` بنویسید.
- نحوهٔ استفاده از `np.add.at` برای جمع‌زدن مقادیر در اندیس‌های مشخص را توضیح دهید.
- مقایسه با استفاده از `scipy.sparse.csr_matrix` و `dot` آن.